



الأنوية المتجانسة (MONOLITHIC) مقابل الأنوية المصغرة (MICROKERNELS)

بحث علمي



١٣ رجب، ١٤٤٧

إعداد الطالب / أمين محمد أمين محسن

تحت إشراف الدكتور / محمد أمين راجع

المحتويات

٤	الفصل الأول: الإطار العام والدراسات النظرية:
٤	1.1 المقدمة:
٤	1.2 أهمية البحث (Research Significance):
٥	1.3 المفاهيم الأساسية لبنية النواة (Kernel Architecture):
٥	1.4 وظائف النواة (Core Function):
٦	1.4 أنواع النواة (Kernel Types):
٦	1.5 برامج تشغيل الاجهزة (Dive Drivers):
٧	1.6 برامج تشغيل الاجهزة (Dive Drivers):
٧	1.7 أمن النواة (Kernel Security):
٨	1.8 آليات التعامل مع الاستدعاءات وسياق التنفيذ (System Calls & Context Switching):
٩	الفصل الثاني: الأنوية المتجانسة (Monolithic Kernels) – معمارية Linux:
٩	2.1.1 مفهوم النواة المصغرة:
٩	2.1.2 العزل والموثوقية:
٩	2.1.3 الأمان وقابلية الصيانة:
٩	2.2 البنية المعمارية لنواة Linux:
١١	2.2 المكونات الأساسية لنواة Linux وآلية تفاعلها:
١٢	2.4 إدارة العمليات والذاكرة في Linux:
١٤	2.5 إدارة الإجهزة وبرامج التشغيل في Linux:
١٥	2.6 أداة Linux وسرعة الاستجابة:
١٥	2.7 الأمان ومساحة الهجوم في Linux:
١٦	2.8 البات وتقنيات الحماية الحديثة في Linux:
١٦	2.9 تقييم معمارية Linux من الناحية الأمنية والأدائية:
١٨	الفصل الثالث: الأنوية الهجينة (Hybrid Kernels) – معمارية Windows:
١٨	3.1 مفهوم الأنوية الهجينة وخصائصها:
١٨	3.2 نشأة وتطور معمارية Windows NT:
١٨	3.4 البنية المعمارية لنواة Windows NT:

١٩	3.5 المكونات الأساسية للنظام وآلية تفاعلها
١٩	3.5 إدارة العمليات والذاكرة في Windows
١٩	3.6 إدارة الأجهزة وبرامج التشغيل
١٩	3.7 أداء Windows وسرعة الاستجابة
٢٠	3.8 الأمان ومساحة الهجوم في Windows
	وتزداد خطورة الثغرات الموجودة في مكونات Kernel Mode بسبب الصلاحيات المرتفعة التي تمتلكها. لذلك يعتمد أمن Windows على العزل بين User Mode و Kernel Mode ، إلى جانب مجموعة من تقنيات الحماية الإضافية.
٢٠	3.9 آليات وتقنيات الحماية الحديثة في Windows
٢٠	3.10 آليات وتقنيات الحماية الحديثة في Windows
٢١	الفصل الرابع: الأنوية المصغرة (Microkernels) — معمارية QNX و Google Fuchsia
٢١	4.1 مفهوم الأنوية المصغرة وخصائصها
٢١	4.2 البنية المعمارية للأنوية المصغرة
٢١	4.3 دراسة حالة: معمارية QNX
٢١	4.4 إدارة العمليات والذاكرة في QNX
٢٢	4.5 الاتصال بين العمليات (IPC) في QNX
٢٢	4.6 الأمان ومساحة الهجوم في QNX
٢٢	4.7 دراسة حالة: Google Fuchsia ونواة Zircon
٢٢	4.8 إدارة العمليات والذاكرة في Zircon
٢٢	4.9 الاتصال بين المكونات في Fuchsia
٢٣	4.10 الأمن القائم على الإمكانات (Capability-Based Security)
٢٣	4.11 الأداء وسرعة الاستجابة في الأنوية المصغرة
٢٣	4.12 التحليل الأمني والأدائي للأنوية المصغرة
٢٤	الفصل الخامس: الدراسة المقارنة والتحليل الأمني والأدائي بين معماريات الأنوية
٢٤	5.1 المقارنة المعمارية بين Linux و Windows و QNX/Fuchsia
٢٤	5.2 مقارنة الأداء وسرعة الاستجابة
٢٤	5.3 مقارنة الأمان وآليات الحماية
٢٤	5.4 مقارنة مساحة الهجوم
٢٤	مقارنة العزل بين مكونات النظام
٢٥	5.5 مقارنة إدارة العمليات والذاكرة
٢٥	5.6 مقارنة آليات الاتصال بين مكونات النظام (IPC)

٢٥	5.7 تحليل نقاط القوة والضعف لكل معمارية.....
٢٥	5.8 النتائج النهائية للمقارنة.....
٢٨	الفصل السادس: المراجع.....

الفصل الأول: الإطار العام والدراسات النظرية:

1.1 المقدمة:

تعتبر أنظمة التشغيل العصب الرئيسي والقلب النابض لكافة البنى الحاسوبية المعاصرة، حيث تقف كحلقة وصل حرجية وحسن تنظيمي منيع بين عتاد الجهاز الصامت وبرمجياته المتقدمة. وفي مركز هذه المنظومة تتربع "النواة" (**Kernel**) ك مكون أساسي يمتلك السلطة المطلقة لإدارة الموارد، وتنظيم الذاكرة، وجدولة المعالجات، والتحكم بالصلاحيات عبر نداءات النظام. ومع الثورة التقنية المتسارعة وانتقال الحوسبة من الأجهزة الشخصية إلى الخوادم السحابية والأنظمة المدمجة الحرجة، أصبحت معمارية النواة تواجه تحدياً هندسياً وأمنياً غير مسبوق؛ يتطلب موازنة دقيقة بين كفاءة الأداء وسرعة الاستجابة من جهة، وتقليص مساحة الهجوم (**Attack Surface**) وعزل الثغرات من جهة أخرى.

تكمن الأهمية الاستراتيجية لهذا البحث في التطور الأمني والهيكل لمعماريات نظم التشغيل، بالتركيز على الصراع الهندسي بين الأنوية المتجانسة (**Monolithic Kernels**) التي تقود أنظمة مثل **Linux**، والأنوية الهجينة (**Hybrid Kernels**) مثل **Windows NT**، والأنوية المصغرة (**Microkernels**) التي تمثل الجيل الحديث في نظم مثل **QNX** و **Google Fuchsia**. يسعى هذا البحث إلى تحليل المفاضلات الأمنية والهندسية بين هذه المعماريات، وكيفية تأثير تصميم النواة على الحد من ثغرات تجاوز الصلاحيات (**Privilege Escalation**)، وحماية الذاكرة، وعزل التعريفات البرمجية، وصولاً إلى استشراف مستقبل بنية نظم التشغيل في البيئات فائقة الحساسية.

1.2 أهمية البحث (Research Significance):

تتجلى أهمية هذا البحث في عدة محاور رئيسية تمس قلب التخصص الأكاديمي والأمني:

\$ الأهمية الأكاديمية والمعمارية:

فهم الآليات العميقة لفصل نمط المستخدم (**User Mode**) عن نمط النواة (**Kernel Mode**)، وكيفية إدارة التبديل السياقي (**Context Switch**) وتأثيره المباشر على الأداء. **\$ الأهمية الأمنية والسيبرانية:**

تحليل تأثير البنية الهندسية على مساحة الهجوم؛ حيث تتيح الأنوية المتجانسة الوصول المباشر للذاكرة مما يزيد خطورة الثغرات، بينما تتبنى الأنوية المصغرة مبدأ الحد الأدنى من الصلاحيات (**Least Privilege**) لتقليص نطاق الأضرار عند حدوث الاختراق. **\$ الأهمية التطبيقية في النظم الحديثة:**

دراسة التطبيقات المعاصرة في الأنظمة ذات الاستجابة اللحظية (**Hard Real-Time Systems**) مثل السيارات ذاتية القيادة والطائرات (**QNX**)، والأجهزة الذكية الحديثة (**Fuchsia**)، وتحديد المعمارية الأنسب لكل بيئة عمل

1.3 المفاهيم الأساسية لبنية النواة (Kernel Architecture):

وهو برنامج أساسي يعمل في قلب كل نظام تشغيل يكمن كوسيط بين برنامج التطبيق ومكونات الأجهزة الخاصة بجهاز الكمبيوتر الخاص بك. إنه مسؤول عن إدارة أجهزة الكمبيوتر (وحدة المعالجة المركزية ، ذاكرة الوصول العشوائي ، التخزين ، إلخ) ، إملأ كيف ومتى يتم استخدام هذه الموارد المختلفة عبر التطبيقات قيد التشغيل والخاملة.

1.4 وظائف النواة (Core Function):

تتمثل وظيفة الكيرنيل الأساسية في ضمان التواصل البرمجيات والأجهزة الفعالة والتي تسمح لها بتشغيل الجهاز الكمبيوتر الخاص بك بسلاسة.

\$ إدارة الموارد:

يتيح الكيرنيل التطبيقات بمشاركة موارد الأجهزة وتنسيق متى وكم مورد أجهزة معينة مطلوبة لمهمه معينه.

\$ وحدة المعالجة المركزية:

يمكن للنواة جدولة عمليات مختلفة لاستخدام وحدة المعالجة المركزية بحيث يتم تشغيل كل شيء بسلاسة وكفاءة مساحة لكل عملية حسب الحاجة ، وتتبع ما هو قيد الاستخدام وما هو مجاني

\$ إدارة العمليات:

تتعامل النواة مع جميع جوانب إدارة العمليات بما في ذلك إنشاء عمليات جديدة وجدولة عند تشغيلها والتنظيف بمجرد الانتهاء منها إنها تتأكد من أن كل عملية لديها الموارد التي تحتاجها للعمل مع حمايتها من التدخل مع بعضها البعض

\$ إدارة الذاكرة:

تشرف النواة على كل من الذاكرة المادية والافتراضية ، مع التأكد من أن كل عملية لديها مساحة كافية للعمل معها يستخدم تقنيات مثل الترحيل (ثابت) والتجزئة (متغير) للحفاظ على كل شيء منظم وفعال

\$ إدارة ملف النظام:

تنظم النواة كيفية تخزين البيانات على القرص الصلب بجهاز الكمبيوتر الخاص بك أو من خلال ضمان حفظ ملفاتك واستردادها بسرعة وأمان.

\$ الأمن والحماية:

تفرض النواة التحكم في الوصول وأذونات لحماية النظام من الوصول غير المصرح به. إنه يعزل العمليات عن بعضها البعض مما يساعد على منع الكود الضار من التأثير على أجزاء أخرى من النظام

1.4 أنواع النواة (Kernel Types):

\$ النواة المتجانسة (Monolithic Kernel):

@ الفكرة والوظيفة: تُصمم كحزمة كبيرة واحدة لا تتضمن فقط الوظائف الأساسية لنظام التشغيل، بل تشمل إدارة نظام الملفات وخدمات النظام الأخرى و أيضاً برامج تشغيل الأجهزة
@ مكان العمل والأداء: تعمل جميع مكوناتها في مساحة عنوان واحدة مما يؤدي إلى أداء عالٍ وكفاءة سرعة جيدة

\$ النواة الدقيقة (Microkernel):

حيث تتعامل فقط مع الوظائف الأكثر أهمية مثل الاتصال بين الأجهزة
@ الفكرة والوظيفة: تعتمد نهج الحد الأدنى حيث تتعامل فقط مع الوظائف الأكثر أهمية مثل الاتصال بين الأجهزة والبرامج وتقويض/تفويض المهام الأخرى (مثل برامج تشغيل الأجهزة) لفصل عمليات وضع المستخدم
@ المميزات: يؤدي هذا التصميم إلى تحسين الاستقرار والنموذجية مما يجعل من السهل عزل وإصلاح مشكلات النظام

\$ النواة الهجينة (Hybrid Kernel):

@ الفكرة والوظيفة: تهدف إلى الحصول على أفضل ما فيها هي مزيج من التصميمات المتجانسة والنواة الدقيقة
@ طريقة العمل: تتميز بخصائص النواة الدقيقة ولكن مع المزيد من النموذجية مثل النواة المتجانسة، مما يجعلها تحافظ على بعض مزايا أداء النواة المتجانسة

\$ النواة المتناهية الصغر (Nanokernel):

@ الفكرة والوظيفة: تهدف للتعامل مع الحد الأدنى المطلق للمهام مقارنة بـ Microkernels وتكون أصغر وأكثر خفة وزن، وتغطي أموراً مثل تبديل السياق وتجريد الأجهزة.
@ الاستخدام: تم تصميمها للتطبيقات المتخصصة، مثل الأنظمة المدمجة أو أجهزة إنترنت الأشياء، حيث تكون البساطة والحد الأدنى من استخدام الموارد هما المفتاح.

\$ إدار النواة الخارجية (Exokernel):

@ الفكرة والوظيفة: مقارنة مختلفة توفر واجهة منخفضة المستوى للأجهزة، مما يتيح للمبرمجين المزيد من التحكم المباشر لاتخاذ القرارات وزيادة التخصص ومكاسب الأداء المحتملة، ولكن يمكن أن يكون معقداً في معالجة الموارد المتاحة.

1.5 برامج تشغيل الاجهزة (Dive Drivers):

تتواصل هذه البرامج مع أجهزة الكمبيوتر لتمكين نظام التشغيل من استخدامها، وتتكون من ٣ أنواع رئيسية:

\$ برامج تشغيل جهاز الأحرف:

@ مسؤوليتها إدارة الأجهزة التي تنقل البيانات حسب حرف الأحرف، بما في ذلك لوحات المفاتيح، الفأرة، والمنفذ التسلسلية.

@ تتفاعل مع عمليات الإدخال والإخراج على أساس الحرف دون تخزين كميات كبيرة من البيانات. ومن الأمثلة عليها: برامج تشغيل لوحة المفاتيح وتفسير ضربات المفاتيح، وبرامج تشغيل الماوس لتتبع حركة المؤشر.

\$ برامج تشغيل جهاز الكتلة:

- @ تشرف على الأجهزة التي تخزن وتسترد البيانات في كتل أو قطاعات ذات حجم ثابت.
- @ تتعامل مع البيانات في أجزاء، مما يتيح قراءة وإعادة كتابة كميات كبيرة من البيانات.
- @ تُستخدم لإدارة أجهزة التخزين وسرعة الأقراص الصلبة، **(SSD)** محركات الأقراص، والتعامل مع طلبات القراءة والكتابة.

\$ برامج تشغيل جهاز الشبكة:

- @ تكون برامج تشغيل شبكة الاتصال **(NIC)** مسؤولة عن التحكم في بطاقات واجهة الشبكة وتدعم الاتصال عبر الشبكة.
- @ تقوم برامج التشغيل هذه بنقل واستقبال حزم البيانات، وتنفذ بروتوكولات الشبكة للتعامل مع تكوين الشبكة وتوجيه الحزم ونقل البيانات.

1.6 برامج تشغيل الأجهزة (Dive Drivers):

\$ وضع النواة: (Kernel Mode)

- @ حالة مميزة تعمل فيها المكونات الأساسية للنظام. يتمتع بوضع الوصول غير المقيد إلى موارد الأجهزة، ويمكنه أداء مهام النظام الحرجة مثل إدارة الذاكرة، والتحكم في الأجهزة، ومقاطع نظام التشغيل.
- @ مستوى المتطلبات: ينطوي على ثقة عالية في المكونات الموثوقة فقط لنظام التشغيل، ويتطلب أداءً سلساً للحفاظ على سلامة النظام.

\$ وضع المستخدم (User Mode):

- @ بيئة مقيدة حيث يتم تشغيل معظم التطبيقات مثل المتصفحات ومعالجات النصوص.
- @ يعمل البرنامج ضمن الحدود التي يحددها نظام التشغيل، دون الوصول المباشر إلى الأجهزة أو موارد النظام الهامة.
- @ تُصمم هذه البيئة لتوفير بيئة آمنة ومستقرة للتطبيقات للعمل بشكل مستقل دون التدخل في الوظائف الأساسية لنظام التشغيل.

1.7 أمن النواة (Kernel Security):

- تحتوي النواة على تدابير أمنية مختلفة لحماية النظام من التهديدات المحتملة وللحفاظ على بيئة آمنة، ومنها:
 - \$ **سياسات التحكم في الوصول:** تحديد أذونات المستخدمين والعمليات؛ حيث تتحقق النواة من صلاحيات الوصول (مثل القراءة والكتابة والتنفيذ) مقابل الأذونات المقترنة بالموارد لضمان أن الكيانات المعتمدة فقط يمكنها الوصول إلى الموارد.
 - \$ **الأذونات والتحقق:** التحقق من حقوق الوصول للكيان المطالب مقابل الأذونات المرتبطة بالموارد، مما يساعد على منع الوصول غير المصرح به وإساءة استخدام موارد النظام.

\$ **تقنيات الرمز / العزل (Sandboxing):** مثل عزل العملية، وفصل مساحة الاسم، وفصل الصلاحيات، لإلقاء بيانات تنفيذ آمنة للتطبيقات لمنعها من الوصول إلى الموارد التابعة للنظام أو تعديلها.

1.8 آليات التعامل مع الاستدعاءات وسياق التنفيذ (System Calls & Context Switching)

Context Switching)

تعتمد النواة في تنظيم العلاقة بين الأجهزة والمستخدم على التجسير الفعال بين وضع المستخدم ووضع النواة، وذلك لضمان تنفيذ العمليات بأعلى قدر من الأمان والكفاءة:

\$ استدعاءات النظام: (System Calls)

تُعد استدعاءات النظام الواجهة الأساسية والوسيط الرئيسي الذي يتيح للتطبيقات التفاعلية العاملة في وضع المستخدم التنسيق مع النواة، وذلك عند رغبتها في طلب تنفيذ مهام حيوية تتطلب صلاحيات مرتفعة (مثل قراءة الملفات من أجهزة التخزين، أو تخصيص مساحات جديدة في الذاكرة العشوائية، أو إنشاء اتصالات عبر برامج تشغيل أجهزة الشبكة). عند إجراء التطبيق لاستدعاء نظام، يتم مقاطعة البرمجية مؤقتاً والانتقال الآمن من وضع المستخدم المقيد إلى وضع النواة ذي الصلاحيات المباشرة.

\$ تبديل السياق: (Context Switching)

عند تنقل المعالج بين إدارة عدة عمليات في وقت واحد، تتدخل النواة لحفظ حالة العملية الحالية (مثل سجلات المعالج ومؤشر التعليمات) بشكل كامل قبل إيقافها، ثم استرجاع حالة العملية التالية لبدء تنفيذها. تضمن هذه الآلية استمرارية العمليات دون فقدان البيانات، وتلعب دوراً جوهرياً في دعم تعدد المهام (Multitasking) وتنظيم عمل أنظمة التشغيل بمختلف معمارياتها (سواء كانت متجانسة أو دقيقة أو هجينة).

\$ إدارة المقاطعات: (Interrupt Handling)

تتولى النواة استقبال الإشارات البرمجية أو المادية (Hardware Interrupts) القادمة من الأجهزة المختلفة (مثل ضربة مفتاح على لوحة المفاتيح أو وصول حزمة بيانات عبر بطاقة الشبكة). عند حدوث المقاطعة، تعلق النواة التنفيذ الحالي مؤقتاً لمعالجة الحدث الطارئ عبر برامج التشغيل المخصصة، ثم تعود لإكمال المهام الرئيسية، مما يعزز الاستجابة اللحظية للنظام وحماية الموارد من التعارض.

نهاية الفصل الأول

الفصل الثاني: الأنوية المتجانسة (Monolithic Kernels) – معمارية

:Linux

2.1.1 مفهوم النواة المصغرة:

تهدف النواة المصغرة (**Microkernel**) إلى تقليل كمية التعليمات البرمجية التي تعمل في وضع الامتيازات (**Privileged Mode**)، بحيث تحتفظ النواة فقط بالخدمات الأساسية الضرورية لتشغيل النظام، مثل الاتصال بين العمليات (**IPC**)، وجدولة العمليات، وإدارة الذاكرة الأساسية. أما الخدمات الأخرى، مثل برامج تشغيل الأجهزة وأنظمة الملفات، فيتم نقلها إلى مساحة المستخدم (**User Space**) بدلاً من تشغيلها داخل النواة نفسها. ومن أمثلة الأنوية المصغرة: **MINIX** و **QNX** و **L4** و **Mach** و **seL4**.

2.1.2 العزل والموثوقية:

من أهم خصائص الأنوية المصغرة الفصل والعزل بين مكونات النظام؛ إذ تعمل العديد من الخدمات في مساحة المستخدم بصلاحيات محدودة، لذلك فإن حدوث خلل في إحدى هذه الخدمات لا يؤدي بالضرورة إلى انهيار النواة الأساسية أو النظام بالكامل. كما يسمح هذا التصميم بإعادة تشغيل الخدمة المتعطلة بشكل مستقل في كثير من الحالات، مما يزيد من استقرار النظام وقدرته على تحمل الأعطال، وهو أمر مهم خصوصاً في الأنظمة التي تتطلب استمرار التشغيل مثل أنظمة السيارات والطيران.

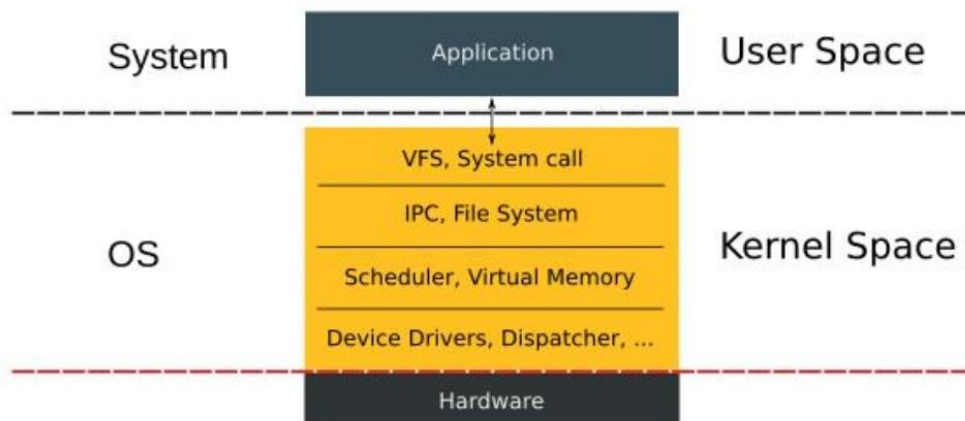
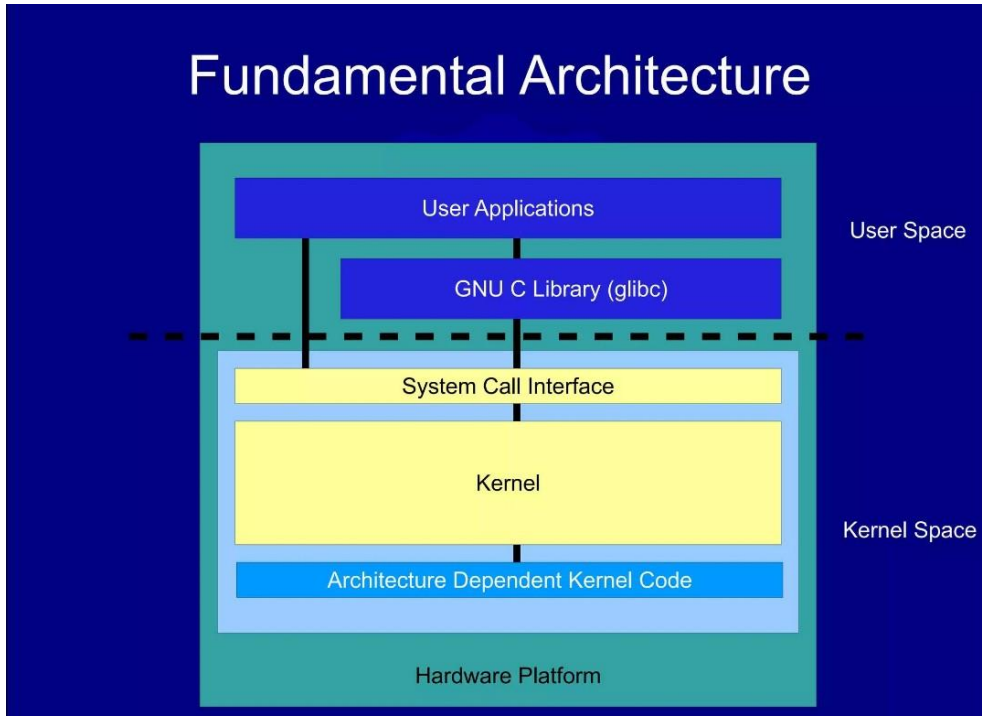
2.1.3 الأمان وقابلية الصيانة:

يساعد صغر حجم النواة على تقليل مساحة الهجوم (**Attack Surface**)، لأن عدد المكونات التي تعمل بصلاحيات عالية داخل **Kernel Space** يكون أقل، بينما يمكن تشغيل المكونات الأخرى بصلاحيات محدودة في **User Space**. كما يتميز التصميم المصغر بكونه أكثر نمطية وأسهل في الصيانة، حيث يمكن تحديث بعض الخدمات أو استبدالها بشكل مستقل دون الحاجة إلى تغيير النواة الأساسية بالكامل. ويظهر هذا النهج بوضوح في الأنظمة التي تركز على الأمان والموثوقية والتحقق الرسمي، مثل **seL4**.

2.2 البنية المعمارية لنواة Linux:

تعتمد **Linux** على النواة المتجانسة (**Monolithic Kernel**)، حيث تعمل معظم خدمات نظام التشغيل الأساسية داخل **Kernel Space** بصلاحيات مرتفعة. وتشمل هذه الخدمات إدارة العمليات والذاكرة، أنظمة الملفات، الشبكات، وبرامج تشغيل الأجهزة. وفي المقابل تعمل التطبيقات في **User Space** بصلاحيات محدودة، وتتواصل مع النواة من خلال **System Calls** عند طلب خدمات منها.

ويتميز **Linux** أيضًا بكونه نواة متجانسة معيارية (**Modular Monolithic Kernel**)، لأنه يسمح بإضافة بعض المكونات مثل برامج تشغيل الأجهزة وأنظمة الملفات كـ **Kernel Modules** دون الحاجة إلى إعادة بناء النواة بالكامل. ويمنح هذا التصميم **Linux** أداءً جيدًا وسرعة في التواصل بين مكونات النواة، لكنه يجعل عددًا كبيرًا من المكونات يعمل بصلاحيات مرتفعة، مما قد يزيد **Attack Surface** عند وجود ثغرات في أحد هذه المكونات.



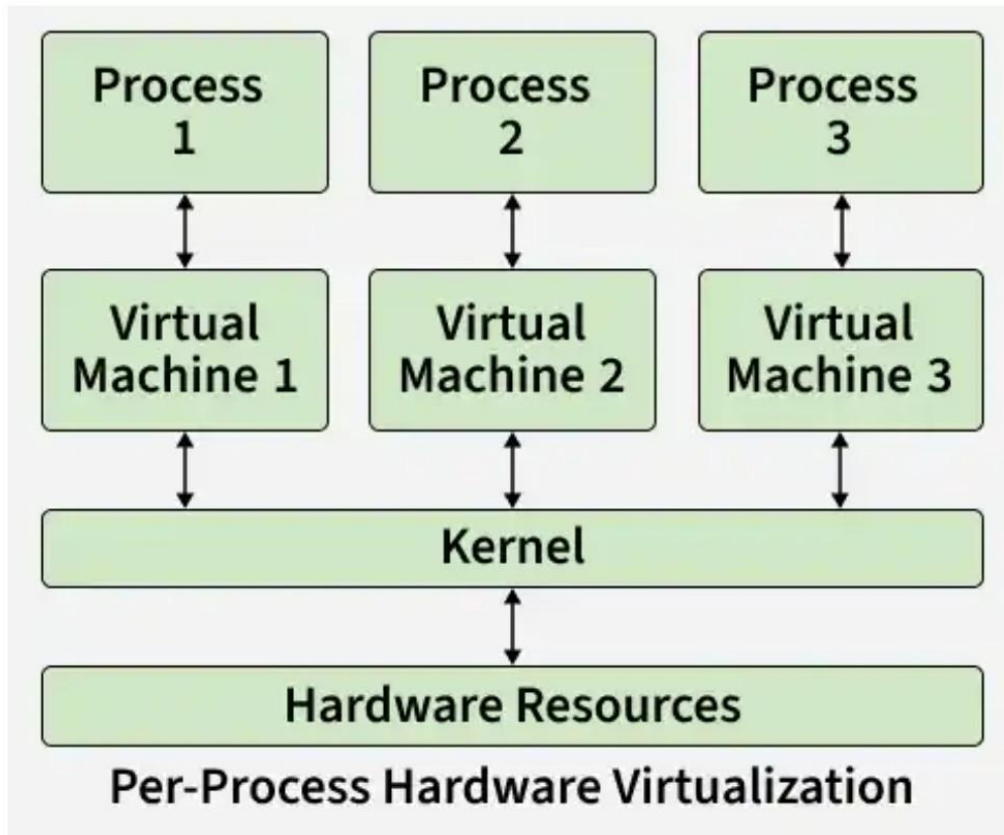
Monolithic Kernel

2.2 المكونات الأساسية لنواة Linux وآلية تفاعلها:

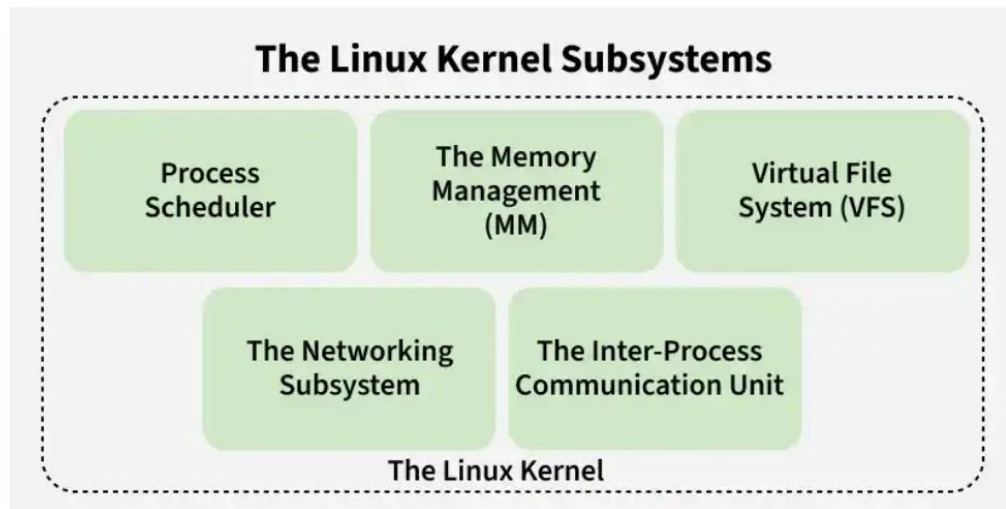
في الحاسوب متعدد الأغراض، تعمل عمليات متعددة في وقت واحد. ولإدارة هذه العمليات بكفاءة، يلزم وجود طبقة برمجية خاصة تُعرف باسم النواة.

تتكون نواة **Linux** من عدة أنظمة فرعية رئيسية، أهمها مُجدول العمليات، وإدارة الذاكرة، ونظام الملفات الافتراضي **VFS**، والنظام الفرعي للشبكات، و**IPC**. يتولى كل نظام فرعي وظيفة محددة؛ فالمُجدول يوزع وقت المعالج بين العمليات، وإدارة الذاكرة توفر الذاكرة وتحميها، بينما يوفر **VFS** طريقة موحدة للتعامل مع الملفات، ويتولى نظام الشبكات الاتصالات، أما **IPC** فيسمح للعمليات بالتواصل والتزامن.

وتتفاعل هذه المكونات مع بعضها أثناء تنفيذ العمليات. فعندما يعمل أحد التطبيقات، يقوم مُجدول العمليات بتحديد وقت استخدامه للـ **CPU**، وتوفر له إدارة الذاكرة مساحة الذاكرة اللازمة، وإذا احتاج إلى قراءة ملف فإنه يتعامل مع **VFS**، وإذا احتاج إلى الاتصال عبر الشبكة يتولى النظام الفرعي للشبكات معالجة الاتصال، بينما يمكن استخدام **IPC** للتواصل مع عمليات أخرى. وبذلك تعمل هذه الأنظمة الفرعية معًا كمنظومة واحدة لإدارة موارد **Linux** وتوفير الخدمات للتطبيقات.



النواة تعمل



الأنظمة الفرعية لنواة لينكس

2.4 إدارة العمليات والذاكرة في Linux :

يُعد نظام **Linux** من أنظمة التشغيل التي تعتمد على النواة لإدارة العمليات والذاكرة والموارد المختلفة بكفاءة. وتُعد إدارة العمليات من الوظائف الأساسية للنواة، حيث تقوم بإنشاء العمليات ومتابعة حالتها وتنظيم تنفيذها على المعالج. وتمثل النواة كل عملية أو خيط تنفيذ باستخدام بنية بيانات تسمى **task_struct**، وهي من أهم هياكل البيانات في **Linux Kernel**.

تحتوي **task_struct** على المعلومات الأساسية الخاصة بالمهمة، مثل معرف العملية (**PID**)، وحالتها الحالية، ومعلومات الجدولة، ومعلومات الإشارات، ومكدس النواة، بالإضافة إلى معلومات مرتبطة بالموارد التي تستخدمها العملية. وتستخدم النواة هذه المعلومات أثناء جدولة العمليات وتبديل التنفيذ بينها. ويمكن أن تكون المهمة عملية كاملة أو خيطاً من خيوط التنفيذ، ولذلك تُعد **task_struct** جزءاً أساسياً من آلية إدارة العمليات والخيوط في **Linux**.

أما الذاكرة الافتراضية (**Virtual Memory**) فهي آلية تسمح للعمليات باستخدام مساحة عناوين افتراضية بدل التعامل مباشرة مع عناوين الذاكرة الفعلية. تحصل كل عملية عادةً على مساحة عناوين افتراضية خاصة بها، مما يساعد على عزل العمليات عن بعضها وحماية الذاكرة من الوصول غير المصرح به. كما تسمح الذاكرة الافتراضية للنظام باستخدام الذاكرة الفعلية بكفاءة، بحيث لا يشترط وجود جميع أجزاء البرنامج في **RAM** في الوقت نفسه.

يدير **Linux** مساحة الذاكرة الافتراضية باستخدام بنية **mm_struct**، التي تحتوي على المعلومات الخاصة بمساحة عناوين العملية. وتنقسم هذه المساحة إلى مناطق مختلفة تسمى **Virtual Memory Areas (VMAs)**، وتمثل كل منطقة نطاقاً من العناوين له خصائص وصلاحيات معينة، مثل القراءة أو الكتابة أو التنفيذ. وتستخدم هذه الآلية لإدارة مناطق البرنامج المختلفة، مثل منطقة الكود والبيانات والمكدس والذاكرة المخصصة ديناميكياً.

يعتمد **Linux** أيضاً على **Paging** لإدارة الذاكرة. تقوم هذه التقنية بتقسيم الذاكرة الافتراضية إلى وحدات ثابتة الحجم تسمى **Pages**، بينما تقسم الذاكرة الفعلية إلى **Page Frames**. ويستخدم النظام **Page Tables** لتخزين معلومات ترجمة العناوين الافتراضية إلى مواقعها في الذاكرة الفعلية. وعندما يحاول البرنامج الوصول إلى عنوان افتراضي، تقوم وحدة إدارة الذاكرة **MMU** الموجودة في المعالج بتنفيذ عملية الترجمة للوصول إلى الموقع الفعلي في **RAM**.

ولتسريع عملية الترجمة، تستخدم المعالجات الحديثة **TLB (Translation Lookaside Buffer)**، وهي ذاكرة تخزين مؤقت تحتفظ بترجمات حديثة للعناوين الافتراضية. وعند العثور على الترجمة المطلوبة في **TLB** تتم عملية الوصول بسرعة أكبر، بينما يؤدي عدم وجودها إلى الحاجة للبحث في **Page Tables**، وهو ما قد يضيف تكلفة زمنية إضافية.

ومن آليات إدارة الذاكرة المهمة في **Linux** أيضاً **Page Fault**. ويحدث عندما تحاول عملية الوصول إلى صفحة غير موجودة حالياً في الذاكرة الفعلية أو عندما يكون الوصول إلى منطقة معينة غير مسموح به. تتدخل النواة لمعالجة الحالة، وقد تقوم بتحميل الصفحة المطلوبة إلى الذاكرة إذا كان الوصول صالحاً، أو تتعامل مع الخطأ إذا كان العنوان أو الصلاحيات غير صحيحة.

كما يستخدم **Linux** تقنية **Copy-on-Write (COW)** لتحسين كفاءة إنشاء العمليات. فعند إنشاء عملية جديدة باستخدام **fork()**، لا تحتاج النواة إلى نسخ جميع صفحات الذاكرة مباشرة، بل يمكن مشاركة الصفحات في البداية، وعندما تحاول إحدى العمليات تعديل صفحة مشتركة يتم إنشاء نسخة خاصة بها. يؤدي ذلك إلى تقليل استهلاك الذاكرة وتحسين سرعة إنشاء العمليات.

وبذلك توفر آليات **Process Management** و **task_struct** و **Virtual Memory** و **Paging** الأساس الذي يسمح لنظام **Linux** بإدارة العمليات والذاكرة بطريقة فعالة وآمنة، مع توفير العزل بين العمليات وتحسين استخدام الذاكرة وتسريع الوصول إليها.

2.5 إدارة الأجهزة وبرامج التشغيل في Linux :

تعتمد نواة **Linux** على معمارية ديناميكية لإدارة الأجهزة وبرامج التشغيل (**Device Drivers**) من خلال ما يُعرف بـ الوحدات النمطية القابلة للتحميل (**Loadable Kernel Modules - LKMs**).
تتيح هذه الميزة للنواة إضافة أو إزالة برامج تشغيل الأجهزة وبروتوكولات الشبكة أثناء التشغيل دون الحاجة لإعادة تشغيل النظام بأكمله أو إعادة ترجمة النواة (**Recompiling**).

المخاطر الأمنية المعمارية لـ **LKMs**:

على الرغم من المرونة العالية التي توفرها وحدات **LKM**، إلا أنها تثير تحديات أمنية جوهرية في معمارية الأنوية المتجانسة:

\$ العمل في نطاق النواة (Ring 0): تعمل جميع وحدات **LKM** المحملة بنفس المستوى العالي من الصلاحيات الذي تمتلكه النواة نفسها. بالتالي، فإن وجود أي أخطاء برمجية أو ثغرة في برنامج تشغيل جهاز متصل بالنظام يمنح المهاجم وصولاً كاملاً للذاكرة وحقوق الجذر (**Root Privileges**).

\$ اتساع مساحة الهجوم (Attack Surface): يأتي توزيع **Linux** افتراضياً بدعم لآلاف الوحدات والتعريفات التي قد لا يحتاجها النظام فعلياً. وجود هذه الوحدات غير المستخدمة يفتح أبواباً لاستغلال ثغرات مثل تصعيد الصلاحيات المحلي (**Local Privilege Escalation**).

\$ إستراتيجيات تقليل المخاطر وتحسين برامج التشغيل:

للمحد من المخاطر المصاحبة لإدارة برامج التشغيل، تتبع النظم الحديثة آليات تحسين محددة:

\$ القائمة البيضاء والتحقق من التوقيع (Module Signing): حظر تحميل أي وحدة **LKM** غير موقعة بنشاط وبمفاتيح تشفير معتمدة من مدير النظام.

\$ إستراتيجية تقليص الوحدات (Attack Surface Reduction): تعطيل وحظر (**Blacklisting**) جميع الوحدات والتعريفات غير الضرورية لعمل الخادم أو الجهاز، والإبقاء فقط على الحد الأدنى المطلوب لتشغيل العتاد المباشر.

\$ أتمتة المراقبة والتدقيق (مثل أداة ModuleJail): استخدام أدوات برمجية لفحص وتحديد الوحدات المحملة بالفعل في الذاكرة ومقارنتها بالقوائم المسموحة لمكافحة التحميل الخبيث أو الثغرات الناتجة عن سوء الإعداد.

2.6 أداة Linux وسرعة الاستجابة :

تؤثر **System Calls** في أداء نواة **Linux** بسبب انتقال التنفيذ من **User Mode** إلى **Kernel Mode**، مما يضيف زمنًا إضافيًا مقارنة باستدعاءات الدوال العادية. وتوضح القياسات أن استخدام **syscall** التقليدي لتنفيذ **getpid()** يسبب تكلفة أعلى من استخدام **vDSO** الذي يسمح بتنفيذ بعض استدعاءات النظام بطريقة أكثر كفاءة. كما يمكن تقليل هذه التكلفة في عمليات الإدخال والإخراج من خلال تجميع عدة عمليات في استدعاء واحد.

أما **Context Switch Overhead** فهو الوقت الذي تحتاجه النواة عند إيقاف تنفيذ عملية أو خيط والانتقال إلى عملية أو خيط آخر، ويتضمن ذلك حفظ حالة التنفيذ الحالية واستعادة حالة التنفيذ التالية. يؤدي تكرار عمليات تبديل السياق بكثرة إلى استهلاك وقت المعالج وزيادة زمن الاستجابة، خصوصًا في الأنظمة التي تحتوي على عدد كبير من العمليات أو الخيوط. لذلك يعتمد أداء **Linux** على تقليل عمليات تبديل السياق غير الضرورية وتحسين آلية جدولة العمليات.

2.7 الأمان ومساحة الهجوم في Linux :

ثغرة **Dirty Frag** وتوسيع مساحة الهجوم
\$ توضح ثغرة **Dirty Frag** أن وجود عيوب منطقية داخل نواة **Linux** يمكن أن يوسع **Attack Surface** ويتيح لمستخدم محلي منخفض الصلاحيات الوصول إلى امتيازات أعلى. تعتمد الثغرة على التفاعل مع آليات **splice** وبعض مكونات الشبكات مثل **xfrm-ESP** و **RxRPC** مما يسمح بالتلاعب ببيانات موجودة في **Page Cache** حتى عندما تكون الملفات على القرص للقراءة فقط.

\$ تأثير الثغرة على **Privilege Escalation**
تكمّن الخطورة في إمكانية استغلال هذا السلوك للتأثير على ملفات نظام حساسة مثل **/etc/passwd** و **/etc/sudoers** أو **/etc/shadow** وهو ما قد يؤدي إلى **Privilege Escalation** والحصول على صلاحيات **Root**. ويزداد الخطر في الخوادم متعددة المستخدمين والبيئات الحاوية، لأن مستخدمًا أو عملية منخفضة الصلاحيات قد تستغل الثغرة للوصول إلى امتيازات أعلى.

2.8 اليات وتقنيات الحماية الحديثة في Linux :

\$ وحماية ذاكرة النواة KASLR

يُعد **KASLR (Kernel Address Space Layout Randomization)** من آليات **Linux Kernel Hardening** التي تهدف إلى جعل استغلال ثغرات الذاكرة أكثر صعوبة. تقوم الفكرة على تغيير أماكن وجود أجزاء مهمة من النواة في الذاكرة بدل جعل عناوينها ثابتة ومتوقعة، وبالتالي إذا حاول المهاجم استغلال ثغرة تعتمد على معرفة عنوان معين داخل النواة، يصبح تحديد العنوان الصحيح أصعب. كما يدعم **Linux** آليات أخرى لتقليل قابلية التنبؤ بذاكرة النواة، مثل عشوائية مناطق الذاكرة وترتيب بعض هياكلها

\$ Lynx SELinux و eBPF كطبقات للتحكم وتقليل المخاطر

يوفر **SELinux (Security-Enhanced Linux)** طبقة أمنية إضافية تعتمد على **Mandatory Access Control (MAC)**، بحيث لا تعتمد حماية النظام فقط على صلاحيات المستخدم التقليدية، بل يمكن تطبيق سياسات تحدد ما الذي يُسمح للعمليات بالوصول إليه وتنفيذه. أما **eBPF** فيوفر إمكانية تشغيل برامج داخل بيئة مرتبطة بالنواة لأغراض مثل المراقبة والشبكات والتتبع، ولذلك يجب التحكم في استخدامه وصلاحياته حتى لا يتحول إلى نقطة إضافية في **Attack Surface** وبشكل عام، تشير منهجية **Kernel Hardening** إلى أن آليات الأمان يجب أن تضيف طبقات حماية، مع تعطيل الإمكانيات غير المطلوبة لتقليل فرص الاستغلال .

Lynx

\$ Namespaces و Cgroups لعزل العمليات وتقليل تأثير الاختراق

تستخدم **Namespaces** لعزل جوانب مختلفة من بيئة العمليات، وهي من الأسس المهمة التي تعتمد عليها الحاويات لعزل العمليات والموارد عن بعضها. أما **Cgroups (Control Groups)** فتستخدم للتحكم في استهلاك الموارد مثل **CPU** والذاكرة والعمليات. الجمع بينهما يساعد على تقليل **Blast Radius** أي أن اختراق عملية أو تطبيق داخل بيئة معزولة لا يعني بالضرورة السيطرة على النظام بأكمله. ومع ذلك، تظل حماية النواة نفسها مهمة، لأن وجود ثغرة في مكون **Kernel** مشترك يمكن أن يسمح للمهاجم بمحاولة تجاوز العزل. لذلك فإن تعطيل المكونات غير المستخدمة، تقليل سطح الهجوم، وتحديث النواة باستمرار تُعد أجزاء أساسية من استراتيجية **Kernel Hardening**.

2.9 تقييم معمارية Linux من الناحية الأمنية والأدائية :

أداء Linux مع المعمارية Monolithic

يعتمد **Linux** على معمارية **Monolithic Kernel**، حيث تعمل مكونات أساسية مثل إدارة العمليات والذاكرة وتعريفات الأجهزة وأنظمة الملفات و **System Calls** ضمن مساحة النواة. يمنح هذا التصميم **Linux** أداءً مرتفعاً؛ لأن مكونات النواة تتواصل مباشرة وبكفاءة، مع عدد أقل من عمليات **Context Switching** وبدون الاعتماد الكبير على **Message Passing** بين خدمات منفصلة. لذلك يتميز **Linux** بسرعة تنفيذ **System Calls** وانخفاض زمن الاستجابة، وهو ما يجعله مناسباً للخوادم وأعمال العمل التي تتطلب أداءً مرتفعاً

DEV Community

\$ التأثير الأمني ومساحة الهجوم

في المقابل، يؤدي جمع عدد كبير من المكونات داخل **Kernel Space** إلى زيادة **Attack Surface** مقارنةً بالـ **Microkernel**. فإذا احتوى أحد مكونات النواة، مثل **Device Driver**، على ثغرة أمنية، فقد تكون آثارها كبيرة؛ لأن المكونات تعمل ضمن بيئة ذات صلاحيات مرتفعة وتشارك في مساحة النواة. وتشير الصفحة إلى أن اختراق أحد مكونات **Monolithic Kernel** قد يؤدي، في أسوأ الحالات، إلى سيطرة واسعة على النظام، بينما يقلل **Microkernel** من هذا الخطر من خلال تشغيل العديد من الخدمات في **User Space** وعزلها عن النواة الأساسية.

DEV Community

\$ دراسة حالة **Linux**: الموازنة بين **Security** و **Performance**

تمثل حالة **Linux** مثالاً عملياً على المقايضة بين **Security** و **Performance**؛ فالمعمارية الأحادية توفر سرعة وتكاملاً عاليين، لكنها تجعل حجم النواة ومساحة الهجوم أكبر، كما أن حدوث خطأ في أحد مكوناتها قد يؤثر في النظام بأكمله. ومع ذلك، اختار **Linux** هذا النهج بصورة عملية لأنه يوازن بين الأداء وسرعة التطوير، مع الاستفادة من **Loadable Kernel Modules** لإضافة قدر من المرونة والتنظيم. وبالتالي فإن **Linux** يوضح أن اختيار **Monolithic Kernel** لا يعني إهمال الأمان، بل يعني قبول قدر من المخاطر المعمارية مقابل تحقيق أداء مرتفع وتكامل قوي بين مكونات النظام.

DEV Community

نهاية الفصل الثاني

الفصل الثالث: الأنوية الهجينة (Hybrid Kernels) — معمارية

Windows

3.1 مفهوم الأنوية الهجينة وخصائصها

النواة الهجينة **Hybrid Kernel** هي تصميم يجمع بين بعض خصائص الأنوية المتجانسة **Monolithic Kernels** ومبادئ الأنوية المصغرة **Microkernels**. وتهدف إلى تحقيق توازن بين الأداء العالي والعزل والتنظيم.

\$ تحتوي الأنوية الهجينة على مكونات أساسية تعمل في **Kernel Mode** لتحقيق سرعة في التواصل، مع استخدام أساليب تنظيم وعزل مستوحاة من الأنوية المصغرة. ومن أهم خصائصها: الأداء الجيد، قابلية التوسع، دعم الوحدات، وتحسين العزل بين بعض مكونات النظام.

\$ تُوصف معمارية **Windows NT** عادةً بأنها معمارية هجينة، مع احتفاظها بعدد من المكونات ذات الامتيازات العالية داخل النواة.

3.2 نشأة وتطور معمارية Windows NT

\$ ظهرت معمارية **Windows NT** بهدف إنشاء نظام تشغيل حديث يتميز بالاستقرار والأمان وقابلية النقل بين منصات المعالجة المختلفة. بدأ تطويرها في الثمانينيات، وارتبط المشروع بفريق هندسي قاده **Dave Cutler**.

\$ صدرت أول نسخة، **Windows NT 3.1**، عام ١٩٩٣، وركزت على تعدد المهام، وإدارة الذاكرة الافتراضية، والحماية بين العمليات، والشبكات وقابلية النقل.

ومع تطور إصدارات **Windows**، أصبحت معمارية **NT** الأساس التقني للإصدارات الحديثة، مع إضافة تقنيات أمنية مثل **Secure Boot** و **VBS** و **HVCI** وغيرها، مما جعل النظام أكثر قدرة على مواجهة التهديدات الحديثة.

3.4 البنية المعمارية لنواة Windows NT

تعتمد **Windows NT** على مجموعة من الطبقات والمكونات التي يمكن تقسيمها بشكل أساسي إلى **User Mode** و **Kernel Mode**.

في **User Mode** تعمل التطبيقات والخدمات التي لا تحتاج إلى امتيازات النواة، مما يساعد على عزلها عن الموارد الحساسة. أما **Kernel Mode** فيضم مكونات ذات صلاحيات مرتفعة، مثل **Executive** و **Kernel** و **HAL** وبرامج التشغيل.

يمكن تبسيط البنية إلى:

Applications → System Calls → Executive/Kernel → Drivers → HAL → Hardware

ويُعد **Executive** مسؤولاً عن العديد من خدمات النظام، مثل إدارة العمليات والذاكرة والإدخال والإخراج والأمان، بينما يتولى **Kernel** الوظائف منخفضة المستوى مثل الجدولة والمقاطعات.

3.5 المكونات الأساسية للنظام وآلية تفاعلها

تتكون **Windows NT** من عدة مكونات مترابطة، أهمها:

- Executive**: يوفر خدمات إدارة العمليات والذاكرة والإدخال والإخراج والأمان.
- Kernel**: يتولى الجدولة والمقاطعات والمزامنة والوظائف منخفضة المستوى.
- Device Drivers**: تربط نظام التشغيل بالأجهزة المختلفة.
- HAL**: توفر طبقة تجريد للتعامل مع تفاصيل العتاد.
- Security Reference Monitor**: يطبق سياسات التحكم بالوصول.
- Object Manager**: يدير موارد النظام باعتبارها كائنات يمكن التحكم في الوصول إليها.

عندما يحتاج أحد التطبيقات إلى خدمة من النظام، ينتقل الطلب عبر واجهات النظام إلى المكونات المناسبة، ويمكن تبسيط المسار إلى:

Application → API/System Call → Executive → Driver → Hardware.

3.5 إدارة العمليات والذاكرة في Windows

يعتمد **Windows** على مفهوم **Process** لتمثيل بيئة تنفيذ البرنامج، بينما تمثل **Threads** وحدات التنفيذ الفعلية داخل العملية. ويقوم نظام التشغيل بجدولة هذه الخيوط وتوزيع وقت المعالج بينها.

أما إدارة الذاكرة فتعتمد على **Virtual Memory**، حيث تحصل العمليات على مساحات عناوين افتراضية منفصلة. ويساعد ذلك على منع العمليات من الوصول المباشر إلى ذاكرة العمليات الأخرى، كما يوفر فصلاً بين مساحة المستخدم والنواة.

وتستخدم **Windows** أيضاً صلاحيات للذاكرة تحدد إمكانية القراءة والكتابة والتنفيذ، مما يساعد في تقليل بعض أنواع استغلال الثغرات.

3.6 إدارة الأجهزة وبرامج التشغيل

\$ تستخدم **Windows** برامج التشغيل **Drivers** للتواصل مع الأجهزة المختلفة مثل الأقراص وبطاقات الشبكة والطابعات وبطاقات الرسومات.

\$ توفر **Windows** تقنيات لإدارة برامج التشغيل والأجهزة، مثل **Plug and Play** وأطر تطوير برامج التشغيل والتوقيع الرقمي.

\$ من الناحية الأمنية، تعد برامج التشغيل مهمة جداً لأنها قد تعمل في **Kernel Mode** وتمتلك صلاحيات مرتفعة. لذلك يمكن أن تؤدي الثغرات الموجودة فيها إلى تأثير كبير على النظام، مما يجعلها عنصراً مهماً عند دراسة مساحة الهجوم.

3.7 أداء Windows وسرعة الاستجابة

تتميز **Windows** بأداء مناسب لأنظمة الاستخدام العام والخوادم، ويرتبط ذلك بعدة عوامل، منها جدولة المعالج وإدارة الذاكرة والإدخال والإخراج ودعم المعالجات متعددة الأنوية.

كما أن وجود بعض المكونات في **Kernel Mode** يقلل بعض تكاليف الاتصال بينها، مما يساعد على تحقيق أداء جيد.

ومع ذلك، تعتمد سرعة الاستجابة الفعلية على عوامل متعددة، مثل العتاد وبرامج التشغيل وعدد العمليات ونوع الحمل. لذلك لا يمكن اعتبار **Windows** أسرع أو أبطأ من نظام آخر دون تحديد معيار القياس ونوع الاستخدام.

3.8 الأمان ومساحة الهجوم في Windows

تمثل مساحة الهجوم **Attack Surface** مجموع المكونات والواجهات التي يمكن أن يستغلها المهاجم للوصول إلى النظام أو التأثير فيه. تحتوي **Windows** على عدد كبير من المكونات والخدمات وبرامج التشغيل وواجهات البرمجة، مثل **Kernel** و **Network Stack** و **File Systems** و **System Services**، مما يجعل إدارة مساحة الهجوم تحديًا مهمًا. وتزداد خطورة الثغرات الموجودة في مكونات **Kernel Mode** بسبب الصلاحيات المرتفعة التي تمتلكها. لذلك يعتمد أمن **Windows** على العزل بين **User Mode** و **Kernel Mode**، إلى جانب مجموعة من تقنيات الحماية الإضافية.

3.9 آليات وتقنيات الحماية الحديثة في Windows

تستخدم **Windows** الحديثة مجموعة من تقنيات الحماية متعددة الطبقات، ومن أهمها: **Secure Boot**: يساعد على حماية عملية الإقلاع ومنع تشغيل مكونات غير موثوقة وفق سياسة النظام. **VBS**: يستخدم المحاكاة الافتراضية لعزل بعض الوظائف الأمنية. **HVCI**: يساعد في حماية سلامة الكود الذي يعمل في **Kernel Mode**. **CFG**: يصعب بعض أساليب استغلال الذاكرة من خلال تقييد مسارات التنفيذ. **DEP**: يمنع تنفيذ التعليمات في مناطق ذاكرة مخصصة للبيانات. **ASLR**: يغير مواقع بعض مكونات الذاكرة لتصعيب استغلال الثغرات. **Microsoft Defender**: يوفر قدرات للكشف عن البرمجيات والتهديدات وحمايتها. وتوضح هذه التقنيات أن أمن **Windows** لا يعتمد على النواة وحدها، بل على دفاع متعدد الطبقات **Defense in Depth**.

3.10 آليات وتقنيات الحماية الحديثة في Windows

\$ من الناحية الأمنية، تتميز **Windows** الحديثة بتعدد طبقات الحماية، مثل **Secure Boot** و **VBS** و **HVCI** و **ASLR** و **DEP** و **CFG**، إضافة إلى العزل بين العمليات ومستويات الصلاحيات. في المقابل، فإن ضخامة النظام وتعدد مكوناته وبرامج التشغيل التي تعمل في **Kernel Mode** تجعل إدارة مساحة الهجوم تحديًا مستمرًا. \$ من الناحية الأدائية، فتميز **Windows** بأداء جيد في أنظمة الاستخدام العام والخوادم، وتدعم تعدد الأنوية والذاكرة الافتراضية وتقنيات الإدخال والإخراج الحديثة. \$ من ناحية سرعة الاستجابة، صُممت **Windows** كنظام تشغيل عام يوازن بين الأداء والتوافق والأمان والاستقرار، وليست موجهة أساسًا إلى أنظمة **Hard Real-Time**. لذلك تختلف نتائج المقارنة حسب نوع الحمل والمعيار المستخدم.

نهاية الفصل الثالث

الفصل الرابع: الأنوية المصغرة (Microkernels) — معمارية QNX

و Google Fuchsia

4.1 مفهوم الأنوية المصغرة وخصائصها

النواة المصغرة **Microkernel** هي تصميم يهدف إلى إبقاء وظائف النواة الأساسية في أصغر حجم ممكن، ونقل معظم خدمات نظام التشغيل إلى **User Space** بدل تشغيلها داخل **Kernel Mode**. تقتصر النواة عادةً على وظائف مثل جدولة العمليات والخيوط، وإدارة الذاكرة الأساسية، والاتصال بين العمليات **IPC**، والتعامل مع المقاطعات. أما الخدمات الأخرى، مثل أنظمة الملفات وبرامج التشغيل وبعض خدمات الشبكة، فتعمل كعمليات منفصلة. يوفر هذا التصميم عزلاً أكبر بين مكونات النظام، لكنه قد يضيف تكلفة إضافية بسبب الاعتماد على **IPC** والتواصل بين العمليات.

4.2 البنية المعمارية للأنوية المصغرة

\$ تتكون **Microkernel** من مجموعة صغيرة من الوظائف الأساسية، أهمها إدارة العمليات والخيوط، وجدولة المعالج، وإدارة الذاكرة الأساسية، والاتصال بين العمليات **IPC**، والتعامل مع المقاطعات. \$ تعمل الخدمات الأخرى كعمليات منفصلة في **User Space**، مثل أنظمة الملفات وبرامج التشغيل وبعض خدمات الشبكة، وتتواصل فيما بينها باستخدام **IPC**. ويؤدي هذا التصميم إلى زيادة العزل، بحيث يمكن أن يقتصر فشل أحد مكونات **User Space** على ذلك المكون بدلاً من التسبب مباشرة في انهيار النواة بالكامل.

4.3 دراسة حالة: معمارية QNX

QNX هو نظام تشغيل تجاري يعتمد على تصميم **Microkernel**، ويستخدم بصورة خاصة في الأنظمة المضمنة **Embedded Systems** والأنظمة التي تتطلب الاستقرار والاستجابة العالية، مثل أنظمة السيارات وبعض الأنظمة الصناعية. \$ يعتمد **QNX** على نواة **QNX Neutrino Microkernel**، وتقتصر مسؤولياتها على الوظائف الأساسية مثل الجدولة، والاتصال بين العمليات، ومعالجة المقاطعات وبعض وظائف إدارة الذاكرة. \$ الخدمات الأخرى فتعمل في **User Space**. ومن المفاهيم المهمة في **QNX Resource Managers**، وهي مكونات توفر واجهات للتعامل مع الموارد والأجهزة وأنظمة الملفات دون الحاجة إلى توسيع النواة بشكل كبير.

4.4 إدارة العمليات والذاكرة في QNX

\$ يعتمد **QNX** على **Processes** و **Threads** لتنفيذ التطبيقات والخدمات، حيث تمثل العملية بيئة تنفيذ معزولة، بينما تمثل الخيوط وحدات التنفيذ التي يقوم النظام بجدولتها. \$ يستخدم **QNX** جدولة تعتمد على الأولويات، مما يساعد على توفير استجابة يمكن التنبؤ بها، وهو أمر مهم في **Real-Time Systems**.

كما يستخدم الذاكرة الافتراضية وآليات العزل لمنع العمليات من الوصول غير المصرح به إلى ذاكرة عمليات أخرى، مما يحسن الاستقرار ويحد من تأثير تعطل أحد المكونات.

4.5 الاتصال بين العمليات (IPC) في QNX

\$ **IPC** يعد عنصرًا أساسيًا في تصميم **QNX** لأن العديد من خدمات النظام تعمل كعمليات مستقلة في **User Space**. ويعتمد **QNX** بصورة رئيسية على **Message Passing** لتبادل الطلبات والبيانات.

\$ تستخدم **Channels** و **Connections** لتنظيم الاتصال بين العمليات، بحيث يمكن لعملية إرسال رسالة إلى أخرى لتنفيذ خدمة أو الحصول على بيانات. يساعد هذا الأسلوب على الحفاظ على العزل بين المكونات، لكنه قد يضيف تكلفة إضافية في بعض الحالات بسبب الاتصال ونقل البيانات بين العمليات.

4.6 الأمان ومساحة الهجوم في QNX

\$ يتميز **QNX** بالعزل بين مكونات النظام، حيث تعمل العديد من الخدمات وبرامج التشغيل خارج النواة في **User Space**. وهذا يقلل كمية الكود الذي يعمل بصلاحيات النواة، ويساعد على تقليل مساحة الهجوم ذات الامتيازات العالية.

كما أن فشل أحد مكونات **User Space** لا يؤدي بالضرورة إلى انهيار النواة، ويمكن في بعض الحالات إعادة تشغيل المكون المتعطل دون إعادة تشغيل النظام بالكامل. ومع ذلك، لا يعني صغر النواة خلو النظام من الثغرات، لأن الخدمات وبرامج التشغيل الموجودة خارج النواة تظل جزءًا من مساحة الهجوم.

4.7 دراسة حالة: Google Fuchsia ونواة Zircon

Fuchsia هو نظام تشغيل طورته **Google** ويعتمد على نواة تسمى **Zircon**. ويهدف إلى توفير منصة حديثة لأجهزة واستخدامات متنوعة، مع التركيز على الأمان والعزل وقابلية التحديث. تعد **Zircon** المكون الأساسي منخفض المستوى في النظام، وتوفر وظائف مثل إدارة العمليات والخيوط، وإدارة الذاكرة، والاتصال بين العمليات، وإدارة الموارد وآليات الأمن القائمة على الإمكانات. وترتبط **Zircon** ببقية مكونات **Fuchsia**، مثل برامج التشغيل والخدمات وبيئات تشغيل التطبيقات، وتعتمد بدرجة كبيرة على العزل ونموذج **Capabilities**.

4.8 إدارة العمليات والذاكرة في Zircon

\$ تستخدم **Zircon** مفاهيم **Processes** و **Threads** و **Jobs** و **Tasks** لإدارة التنفيذ. فالـ **Process** يمثل مساحة تنفيذ معزولة، بينما يمثل **Thread** وحدة التنفيذ، وتستخدم **Jobs** لتنظيم العمليات والخيوط وبعض الصلاحيات والموارد.

\$ تعتمد **Zircon** على إدارة الذاكرة الافتراضية، إضافة إلى **Virtual Memory Objects (VMOs)** لإدارة مناطق الذاكرة ومشاركتها بطريقة منظمة. يساعد هذا التصميم على توفير عزل قوي بين العمليات، مع المحافظة على مستوى جيد من الاستقرار والأداء وفقًا لطبيعة الحمل.

4.9 الاتصال بين المكونات في Fuchsia

يعتمد **Fuchsia** بصورة كبيرة على **IPC** لفصل المكونات والخدمات عن بعضها، وتستخدم **Channels** كوسيلة أساسية للتواصل وتبادل الرسائل بين العمليات والمكونات. يمكن لمكون إرسال طلب إلى مكون آخر عبر قناة والحصول على النتيجة دون الحاجة إلى دمج المكونين في

ويحسن ذلك العزل ويتيح فصل الخدمات، لكن كثرة الاتصالات ونقل البيانات قد تضيف تكلفة إضافية، لذلك يعتمد تأثير **IPC** على تصميم التطبيق وطبيعة الحمل.

4.10 الأمن القائم على الإمكانيات (Capability-Based Security)

\$ تعتمد **Zircon** على نموذج أمني قائم على **Capabilities**، ويهدف إلى التحكم الدقيق في وصول العمليات إلى موارد النظام. فلا يكفي وجود العملية للحصول على المورد، بل يجب أن تمتلك القدرة أو المرجع المناسب.

\$ تستخدم **Zircon Handles** لتمثيل مراجع إلى موارد النظام مع صلاحيات مرتبطة بها، ويرتبط ذلك بمبدأ **Least Privilege** الذي يمنح العملية أقل قدر ممكن من الصلاحيات اللازمة.

يساعد هذا النموذج على عزل التطبيقات والخدمات وتقليل تأثير اختراق أحد المكونات، وبالتالي الحد من مساحة الهجوم.

4.11 الأداء وسرعة الاستجابة في الأنوية المصغرة

\$ قد يؤدي تصميم **Microkernel** إلى زيادة بعض تكاليف الاتصال مقارنة بالتصميم المتجانس، لأن الخدمات تعمل غالبًا في عمليات منفصلة وتتواصل باستخدام **IPC**.

\$ تتأثر سرعة الاستجابة بعوامل مثل تكلفة **IPC**، وعدد عمليات الانتقال بين المكونات، وجدولة المهام، وكمية البيانات المتبادلة.

لكن العزل لا يعني بالضرورة أداءً ضعيفًا؛ فقد صممت أنظمة مثل **QNX** لتحقيق استجابة عالية وقابلة للتنبؤ، خصوصًا في الأنظمة الزمنية. ولذلك يجب تحقيق توازن بين الأمان والعزل من جهة والأداء وسرعة الاستجابة من جهة أخرى.

4.12 التحليل الأمني والأدائي للأنوية المصغرة

تتميز **Microkernels** بصغر حجم النواة والعزل بين الخدمات وتقليل عدد المكونات التي تعمل

\$ بصلاحيات **Kernel Mode**. كما يمكن أن يكون تأثير فشل أحد مكونات **User Space** محدودًا مقارنةً بمكون يعمل داخل النواة.

لكنها تواجه تحديات، أهمها الاعتماد الكبير على **IPC** واحتمال زيادة تكلفة الاتصال بين المكونات في بعض الأحمال. كما أن تقليل حجم النواة لا يلغي مساحة الهجوم بالكامل، لأن الخدمات وبرامج التشغيل تبقى مكونات تحتاج إلى الحماية.

وبالنسبة إلى **QNX** و **Fuchsia**، يشترك النظامان في التركيز على العزل و **IPC**، بينما يتميز **QNX** بتركيزه القوي على الأنظمة المضمنة و **Real-Time**، ويبرز **Fuchsia/Zircon** في نموذج الأمان القائم على **Capabilities**. ويمهد ذلك للمقارنة الشاملة في الفصل الخامس من حيث الأمان ومساحة الهجوم والأداء وسرعة الاستجابة.

نهاية الفصل الرابع

الفصل الخامس: الدراسة المقارنة والتحليل الأمني والأدائي بين معماريات الأنوية

5.1 المقارنة المعمارية بين Linux و Windows و QNX/Fuchsia

\$ تختلف الأنظمة الثلاثة في طريقة توزيع وظائف نظام التشغيل. يعتمد **Linux** بصورة أساسية على نموذج **Monolithic Kernel**، حيث توجد مجموعة كبيرة من الخدمات وبرامج التشغيل داخل **Kernel Space** مع إمكانية تحميل بعض المكونات كوحدات.

\$ أما **Windows NT** فيُوصف عادةً بأنه **Hybrid Kernel**، إذ يجمع بين بعض خصائص التصميم المتجانس ومبادئ التنظيم والعزل، مع وجود مكونات عديدة ذات امتيازات مرتفعة.

\$ بينما يعتمد **QNX** على **Microkernel**، وتستخدم **Fuchsia** نواة **Zircon** ذات تصميم **Microkernel**، حيث تقتصر النواة على الوظائف الأساسية وتعمل خدمات عديدة في **User Space**. ويؤثر هذا الاختلاف مباشرة في العزل ومساحة الهجوم والأداء.

5.2 مقارنة الأداء وسرعة الاستجابة

\$ يوفر **Linux** أداءً مرتفعاً في الاستخدامات العامة والخوادم نتيجة قرب العديد من مكونات النظام من النواة، كما يمكن تحسينه وفق طبيعة الحمل.

\$ يقدم **Windows** أداءً جيداً في الاستخدام العام والخوادم، مع تركيز على التوافق وتعدد المهام وإدارة الموارد.

\$ أما **QNX** فيتميز بالاستجابة القابلة للتنبؤ، خصوصاً في الأنظمة الزمنية، بينما يعتمد أداء **Fuchsia/Zircon** على تصميم المكونات وعمليات **IPC**. وقد تضيق **Microkernels** تكلفة اتصال إضافية، لكنها توفر في المقابل عزلاً أكبر.

5.3 مقارنة الأمان وآليات الحماية

\$ يوفر **Linux** آليات متعددة للحماية، مثل صلاحيات المستخدمين، العزل بين العمليات، وآليات التحكم بالوصول، إضافة إلى تقنيات مثل **SELinux** و **AppArmor** في الأنظمة التي تستخدمها.

\$ يوفر **Windows** طبقات حماية متعددة، منها **Secure Boot** و **VBS** و **HVCI** و **ASLR** و **CFG** و **DEP**، إضافة إلى آليات التحكم في الصلاحيات والحماية من التهديدات.

\$ أما **QNX** فيستفيد من صغر النواة والعزل بين الخدمات، بينما يركز **Fuchsia/Zircon** على نموذج **Least Privilege Capabilities** للتحكم الدقيق في الوصول إلى الموارد. لذلك تختلف فلسفة الحماية بين الأنظمة، لكن جميعها تعتمد على أكثر من آلية واحدة.

5.4 مقارنة مساحة الهجوم

\$ تتأثر مساحة الهجوم بعدد المكونات والواجهات التي يمكن الوصول إليها، وبمستوى الصلاحيات التي تعمل به هذه المكونات.

\$ يمتلك **Linux** مساحة هجوم واسعة نسبياً بسبب حجم النظام وتعدد برامجه التشغيلية ومكوناته، بينما يحتوي **Windows** أيضاً على عدد كبير من الخدمات والواجهات وبرامج التشغيل.

\$ في **QNX** و **Fuchsia/Zircon**، يساعد وضع عدد أكبر من الخدمات خارج النواة على تقليل مساحة الهجوم ذات الامتيازات العالية، لكنه لا يلغي مساحة الهجوم بالكامل، لأن خدمات **User Space** وبرامج التشغيل ما تزال بحاجة إلى الحماية.

مقارنة العزل بين مكونات النظام

في **Linux** توجد حدود عزل بين العمليات ومساحة المستخدم والنواة، إلا أن عددًا كبيرًا من المكونات يعمل داخل **Kernel Space**.

في **Windows** يوجد فصل واضح بين **User Mode** و **Kernel Mode**، مع تقنيات إضافية للعزل والحماية.

أما **Microkernels** مثل **QNX** و **Zircon** فتجعل العزل جزءًا أساسيًا من التصميم، إذ تعمل خدمات عديدة كعمليات منفصلة في **User Space**. وهذا قد يحد من تأثير فشل أو اختراق أحد المكونات مقارنةً بمكون يعمل داخل النواة.

5.5 مقارنة إدارة العمليات والذاكرة

\$ تستخدم الأنظمة الثلاثة مفهوم العمليات والخيوط مع اختلافات في التنفيذ. يعتمد **Linux** و **Windows** على عمليات ذات مساحات عناوين افتراضية وخيوط يتم جدولتها بواسطة النظام.

\$ يعتمد **QNX** أيضًا على **Processes** و **Threads** مع جدولة تعتمد على الأولويات، وهو ما يناسب الأنظمة الزمنية.

\$ أما **Zircon** فتستخدم **Processes** و **Threads** و **Jobs** و **Tasks**، إلى جانب **Virtual Memory Objects (VMOs)**. وتوفر الأنظمة الثلاثة آليات للعزل وحماية الذاكرة، لكن تختلف طريقة تنظيمها بحسب المعمارية.

5.6 مقارنة آليات الاتصال بين مكونات النظام (IPC)

\$ يستخدم **Linux** مجموعة من آليات **IPC**، مثل **Pipes** و **Sockets** و **Shared Memory** وغيرها، إضافة إلى آليات الاتصال الخاصة بمكونات النظام.

\$ يوفر **Windows** مجموعة من آليات الاتصال بين العمليات، مثل **RPC** و **ALPC** وغيرها، بما يسمح للخدمات والعمليات بتبادل البيانات.

\$ أما **QNX** فيعتمد بصورة أساسية على **Message Passing Channels** و **Connections**، بينما تستخدم **Zircon Channels** كآلية رئيسية للتواصل بين المكونات. ويزداد دور **IPC** في **Microkernels** بسبب وجود الخدمات في **User Space**.

5.7 تحليل نقاط القوة والضعف لكل معمارية

\$ يتميز **Linux** بالأداء والمرونة ودعم عدد كبير من الأجهزة والبرمجيات، لكن حجم النواة وتعدد المكونات ذات الامتيازات العالية يمثلان تحديًا أمنيًا.

\$ يتميز **Windows** بالتوازن بين الأداء والتوافق والأمان، ويقدم طبقات حماية متقدمة، لكن تعقيد النظام وكثرة الخدمات وبرامج التشغيل يزيدان من تحديات إدارة مساحة الهجوم.

\$ يتميز **QNX** بالعزل والاستجابة القابلة للتنبؤ، خصوصًا في الأنظمة المضمنة، بينما يركز

Fuchsia/Zircon على العزل والأمان القائم على **Capabilities**. ومن التحديات في الأنوية المصغرة تكلفة **IPC** وتعقيد إدارة الخدمات المنفصلة.

5.8 النتائج النهائية للمقارنة

توضح المقارنة أن اختيار معمارية النواة يعتمد على طبيعة النظام وأولوياته. يوفر **Linux** و **Windows** أداءً قويًا وتوافقًا واسعًا، بينما تركز **QNX** و **Zircon** بدرجة أكبر على العزل وتقليل امتيازات النواة. من الناحية الأمنية، يساعد التصميم المصغر على تقليل حجم النواة وتقليل تأثير فشل بعض الخدمات، في حين تعتمد **Linux** و **Windows** على مجموعة واسعة من آليات الحماية داخل بنية أكبر وأكثر تعقيدًا.

أما من ناحية سرعة الاستجابة، فيتميز **QNX** خصوصًا في الأنظمة الزمنية التي تتطلب استجابة قابلة للتنبؤ، بينما توفر **Linux** و **Windows** أداءً مناسبًا لمجموعة واسعة من الاستخدامات. ويظل **Fuchsia/Zircon** تصميمًا حديثًا يركز على العزل والأمان مع الاعتماد على **IPC**. وبصورة عامة، لا توجد معمارية واحدة تتفوق في جميع المعايير؛ إذ يمثل الاختيار مفاضلة بين الأداء، العزل، الأمان، التوافق، وحجم مساحة الهجوم.

المحور	QNX / Fuchsia	Windows NT	Linux
نوع المعمارية	Microkernel	Hybrid	Monolithic
مكان الخدمات	User Space عدد كبير في	Kernel/User مزيج من Mode	Kernel Space عدد كبير داخل
الأداء	IPC جيد، مع تكلفة محتملة	جيد إلى مرتفع	مرتفع
سرعة الاستجابة	ممتازة وقابلة QNX Real-Time؛ Zircon تعتمد على الحمل	جيدة للاستخدام العام	جيدة وتختلف حسب الحمل
الأمان	+ عزل قوي في Capabilities Zircon	متعدد الطبقات	متعدد الطبقات
مساحة الهجوم	Kernel أقل في نسبيًا Mode	واسعة نسبيًا	واسعة نسبيًا
العزل	قوي بين الخدمات	مع User/Kernel جيد بين تقنيات إضافية	جيد بين العمليات والمستخدم/النواة
إدارة العمليات	QNX: Processes / Threads؛ Zircon: Processes / Threads / Jobs	Processes / Threads	Processes / Threads
إدارة الذاكرة	Virtual Memory في VMOS و Zircon	Virtual Memory	Virtual Memory
IPC	QNX: Message Passing؛ Zircon: Channels	وغيرها RPC / ALPC	

-	التوازن بين الأداء والأمان والتوافق	العزل وتقليل امتيازات النواة	أبرز نقطة قوة
-	تعقيد النظام وكثرة المكونات	وتعقيد IPC تكلفة الخدمات	أبرز تحدٍ

نهاية الفصل الخامس

الفصل السادس: المراجع

Hostwinds

DEV : <https://dev.to/adityabhuyan/monolithic-kernel-vs-microkernel-understanding-the-key-trade-offs-in-modern-operating-systems-23ln>

Geeksforgeeks: <https://www.geeksforgeeks.org/linux-unix/the-linux-kernel/>

Medium: <https://sourav-k-paul.medium.com/how-linux-manages-memory-from-virtual-addresses-to-huge-pages-a9de33fff8ac>

arkanis: <https://arkanis.de/weblog/2017-01-05-measurements-of-system-call-performance-and-overhead/>

nsfocusglobal: <https://nsfocusglobal.com/linux-kernel-privilege-escalation-vulnerability-dirty-frag-alert/>

lynx software technology: <https://www.lynx.com/blog/securing-your-linux-configuration-kernel-hardening>